

# GPU

## Architektury Komputerów – Wykład 10

Kuba Nowak

Uniwersytet Wrocławski

27 kwietnia 2025

## Computer Architecture

### Lecture 1: Pipelining

Author: Prof. Onur Mutlu  
(modified by Krystian Baclawski)

ETH Zürich  
Spring 2021  
16 April 2021

## Computer Architecture

### Lecture 9: Graphics Processing Units

Authors: Dr. Juan Gómez Luna, Prof. Onur Mutlu  
(modified by Krystian Baclawski)

ETH Zürich  
Spring 2021  
20 May 2021

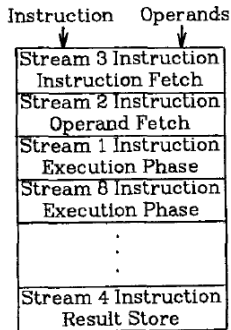
## Computer Architecture

### Lecture 31: GPU Programming

Dr. Juan Gómez Luna  
Prof. Onur Mutlu  
ETH Zürich  
Fall 2024  
9 January 2025

# Fine-Grained Multithreading

- Idea: Hardware has multiple thread contexts (PC+registers). Each cycle, fetch engine fetches from a different thread.
  - ❑ By the time the fetched branch/instruction resolves, no instruction is fetched from the same thread
  - ❑ Branch/instruction resolution latency overlapped with execution of other threads' instructions
- + No logic needed for handling control and data dependences within a thread
- Single thread performance suffers
- Extra logic for keeping thread contexts
- Does not overlap latency if not enough threads to cover the whole pipeline



## Fine-Grained Multithreading (II)

---

- Idea: Switch to another thread every cycle such that no two instructions from a thread are in the pipeline concurrently
- Tolerates the control and data dependency latencies by overlapping the latency with useful work from other threads
- Improves pipeline utilization by taking advantage of multiple threads
- Thornton, “Parallel Operation in the Control Data 6600,” AFIPS 1964.
- Smith, “A pipelined, shared resource MIMD computer,” ICPP 1978.

# Fine-Grained Multithreading

---

## ■ Advantages

- + No need for dependency checking between instructions  
(only one instruction in pipeline from a single thread)
- + No need for branch prediction logic
- + Otherwise-bubble cycles used for executing useful instructions from different threads
- + Improved system throughput, latency tolerance, utilization

## ■ Disadvantages

- Extra hardware complexity: multiple hardware contexts (PCs, register files, ...), thread selection logic
- Reduced single thread performance (one instruction fetched every N cycles from the same thread)
- Resource contention between threads in caches and memory
- Some dependency checking logic *between* threads remains (load/store)

# GPUs are SIMD Engines Underneath

---

- The **instruction pipeline** operates like a **SIMD pipeline** (e.g., an array processor)
- However, the **programming is done using threads**, NOT SIMD instructions
- To understand this, let's go back to our parallelizable code example
- But, before that, let's distinguish between
  - **Programming Model (Software)**
  - vs.
  - **Execution Model (Hardware)**

# Programming Model vs. Hardware Execution Model

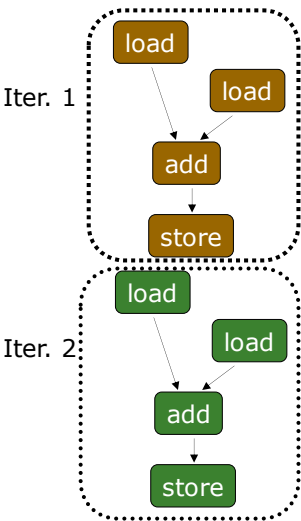
---

- Programming Model refers to **how the programmer expresses the code**
  - ❑ E.g., Sequential (von Neumann), Data Parallel (SIMD), Dataflow, Multi-threaded (MIMD, SPMD), ...
- Execution Model refers to **how the hardware executes the code underneath**
  - ❑ E.g., Out-of-order execution, Vector processor, Array processor, Dataflow processor, Multiprocessor, Multithreaded processor, ...
- **Execution Model can be very different from the Programming Model**
  - ❑ E.g., von Neumann model implemented by an OoO processor
  - ❑ E.g., SPMD model implemented by a SIMD processor (a GPU)

# How Can You Exploit Parallelism Here?

```
for (i=0; i < N; i++)  
  C[i] = A[i] + B[i];
```

*Scalar Sequential Code*



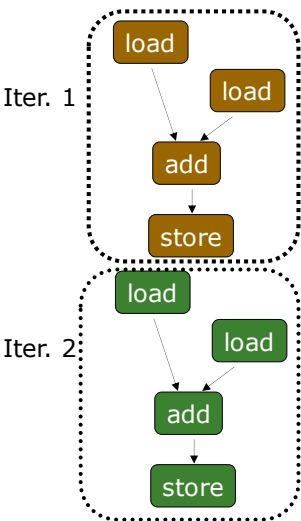
Let's examine three programming options to exploit **instruction-level parallelism** present in this sequential code:

1. Sequential (SISD)
2. Data-Parallel (SIMD)
3. Multithreaded (MIMD/SPMD)

# Prog. Model 1: Sequential (SISD)

```
for (i=0; i < N; i++)  
    C[i] = A[i] + B[i];
```

## Scalar Sequential Code



- Can be executed on a:
  - Pipelined processor
  - Out-of-order execution processor
    - ❑ Independent instructions executed when ready
    - ❑ Different iterations are present in the instruction window and can execute in parallel in multiple functional units
    - ❑ In other words, the loop is dynamically unrolled by the hardware
- Superscalar or VLIW processor
  - ❑ Can fetch and execute multiple instructions per cycle

# Prog. Model 2: Data Parallel (SIMD)

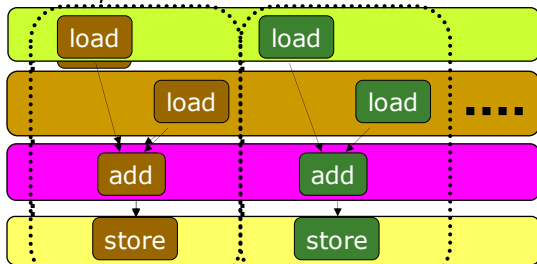
```
for (i=0; i < N; i++)  
    C[i] = A[i] + B[i];
```

Scalar Sequential Code

Vector Instruction

Vectorized Code

Iter.



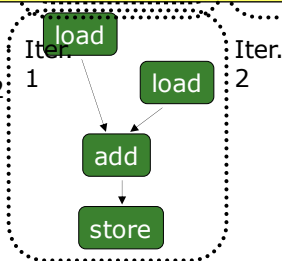
VLD A → V1

VLD B → V2

VADD V1 + V2 → V3

VST V3 → C

Iter. 2



Realization: Each iteration is independent

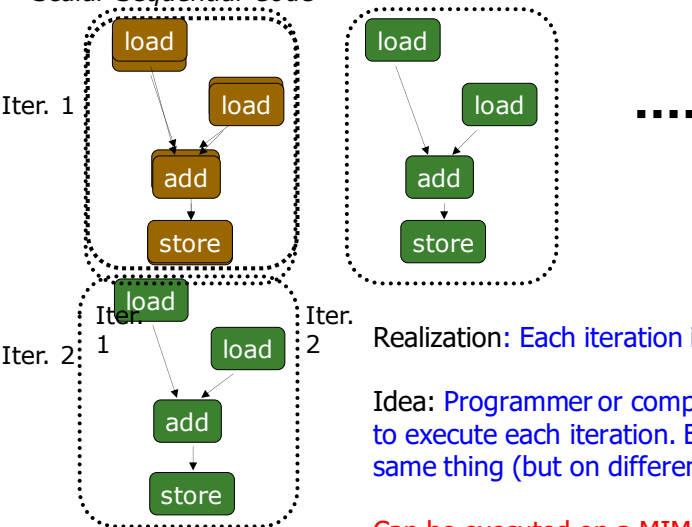
Idea: Programmer or compiler generates a SIMD instruction to execute the same instruction from all iterations across different data

Best executed by a SIMD processor (vector, array)

# Prog. Model 3: Multithreaded

```
for (i=0; i < N; i++)  
    C[i] = A[i] + B[i];
```

## Scalar Sequential Code



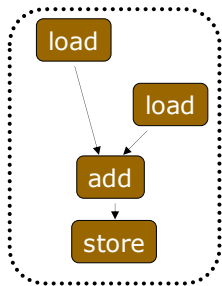
Realization: Each iteration is independent

Idea: Programmer or compiler generates a thread to execute each iteration. Each thread does the same thing (but on different data)

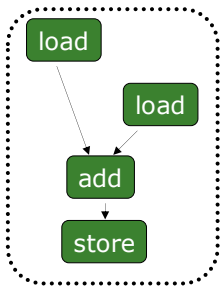
Can be executed on a MIMD machine

# Prog. Model 3: Multithreaded

```
for (i=0; i < N; i++)  
    C[i] = A[i] + B[i];
```



Iter.  
1



Iter.  
2

...

Realization: Each iteration is independent

This particular model is also called:

SPMD: Single Program Multiple Data

Can be executed on a SMT machine  
Single Instruction Multiple Thread

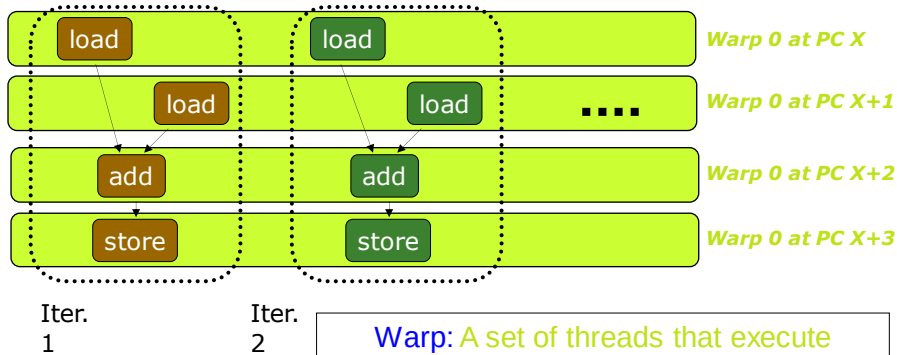
# A GPU is a SIMD (SIMT) Machine

---

- Except it is **not** programmed using SIMD instructions
- It is **programmed using threads** (SPMD programming model)
  - Each thread executes the same code but operates a different piece of data
  - Each thread has its own context (i.e., can be treated/restarted/executed independently)
- A set of threads executing the same instruction are dynamically grouped into a **warp (wavefront)** by the hardware
  - A warp is essentially a **SIMD operation formed by hardware!**

# SPMD on SIMT Machine

```
for (i=0; i < N; i++)  
    C[i] = A[i] + B[i];
```



**Warp:** A set of threads that execute the same instruction (i.e., at the same PC)

This particular model is also called:

**SPMD: Single Program Multiple Data**

A GPU executes it using the SIMT model:  
**Single Instruction Multiple Thread**

# Graphics Processing Units

## SIMD not Exposed to Programmer (SIMT)

# SIMD vs. SIMT Execution Model

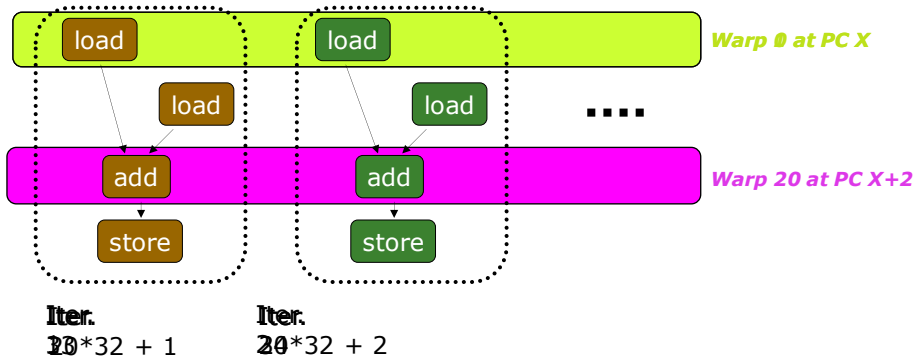
---

- SIMD: A single **sequential instruction stream** of SIMD **instructions** → each instruction specifies multiple data inputs
  - [VLD, VLD, VADD, VST], VLEN
- SIMT: **Multiple instruction streams** of **scalar instructions** → threads grouped dynamically into warps
  - [LD, LD, ADD, ST], NumThreads
- Two Major SIMT Advantages:
  - **Can treat each thread separately** → i.e., can execute each thread independently (on any type of scalar pipeline) → MIMD processing
  - **Can group threads into warps flexibly** → i.e., can group threads that are supposed to *truly* execute the same instruction → dynamically obtain and maximize benefits of SIMD processing

# Fine-Grained Multithreading of Warps

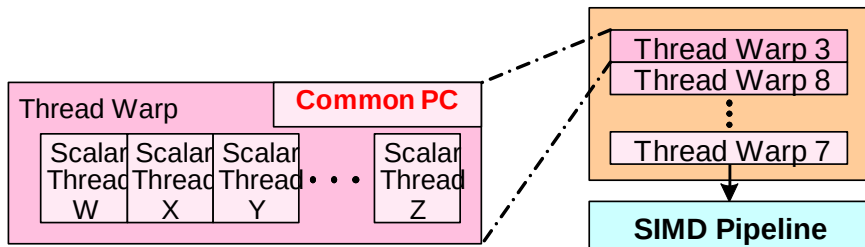
```
for (i=0; i < N; i++)  
    C[i] = A[i] + B[i];
```

- Assume a warp consists of 32 threads
- If you have 32K iterations, and 1 iteration/thread  $\rightarrow$  1K warps
- Warps can be interleaved on the same pipeline  $\rightarrow$  Fine grained multithreading of warps

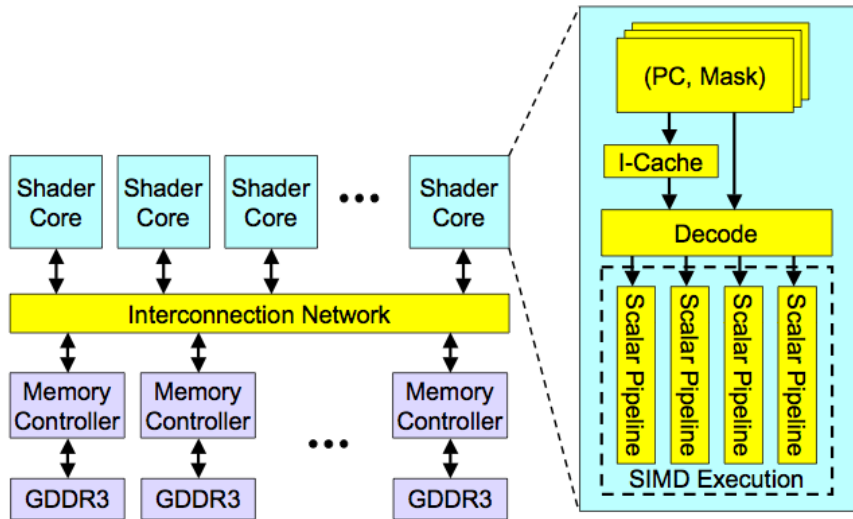


# Warps and Warp-Level FGMT

- Warp: A set of threads that execute the same instruction (on different data elements) → SIMT (Nvidia-speak)
- All threads run the same code
- Warp: The threads that run lengthwise in a woven fabric ...

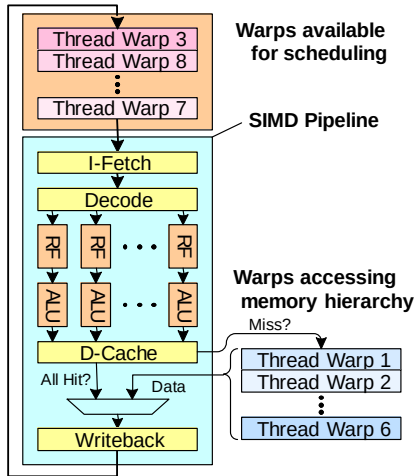


# High-Level View of a GPU



# Latency Hiding via Warp-Level FGMT

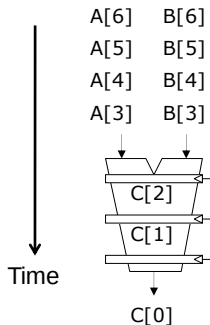
- Warp: A set of threads that execute the same instruction (on different data elements)
- Fine-grained multithreading
  - ❑ One instruction per thread in pipeline at a time (No interlocking)
  - ❑ Interleave warp execution to hide latencies
- Register values of all threads stay in register file
- FGMT enables long latency tolerance
  - ❑ Millions of pixels



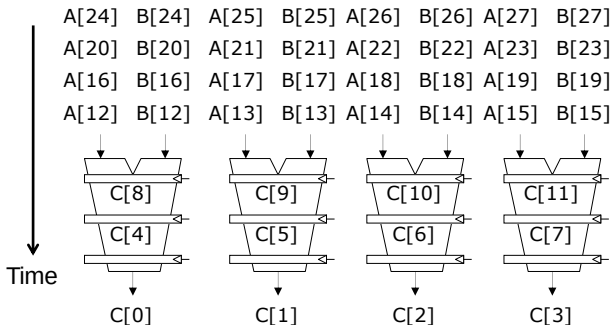
# Warp Execution (Recall the Slide)

32-thread warp executing  $\text{ADD } A[\text{tid}], B[\text{tid}] \rightarrow C[\text{tid}]$

*Execution using  
one pipelined  
functional unit*

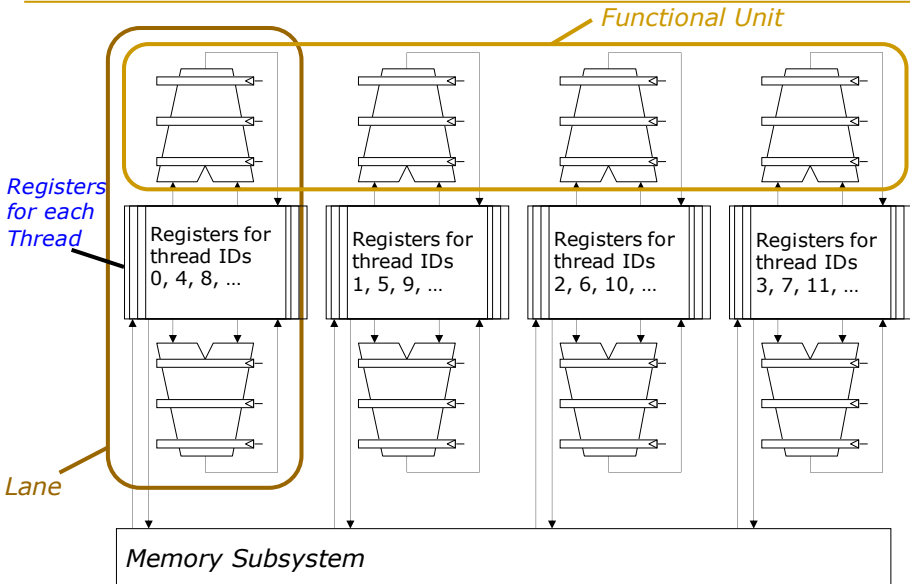


*Execution using  
four pipelined  
functional units*



← Space →

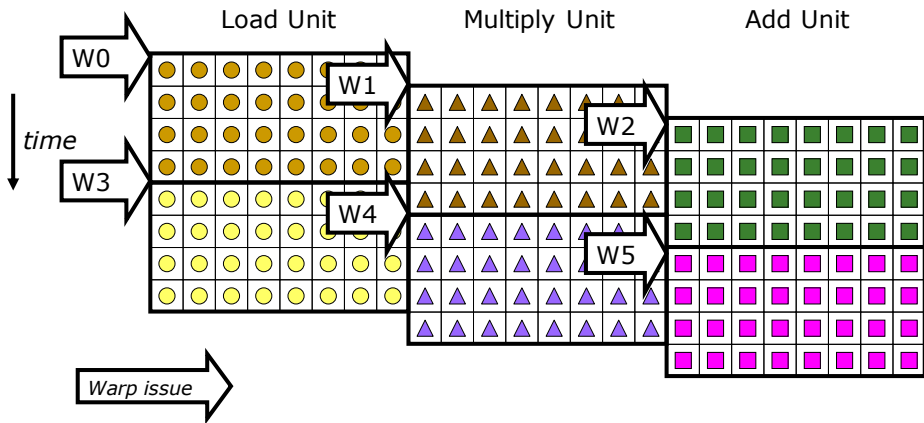
# SIMD Execution Unit Structure



# Warp Instruction Level Parallelism

Can overlap execution of multiple instructions

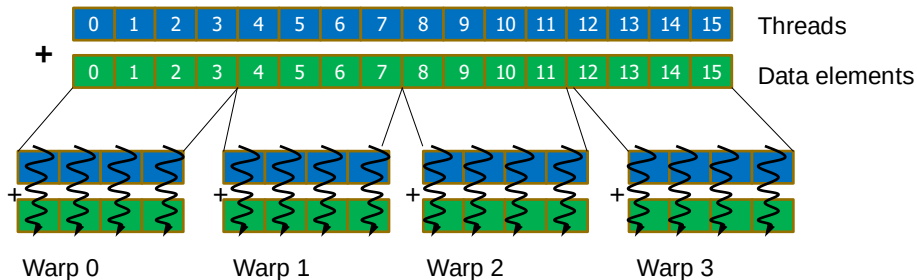
- Example machine has 32 threads per warp and 8 lanes
- Completes 24 operations/cycle while issuing 1 warp/cycle



# SIMT Memory Access

- Same instruction in different threads uses **thread id** to index and access different data elements

Let's assume  $N=16$ , 4 threads per warp  $\rightarrow$  4 warps



# Warps *not* Exposed to GPU Programmers

- CPU threads and GPU kernels
  - Sequential or modestly parallel sections on CPU
  - Massively parallel sections on GPU: Blocks of threads

Serial Code (host)

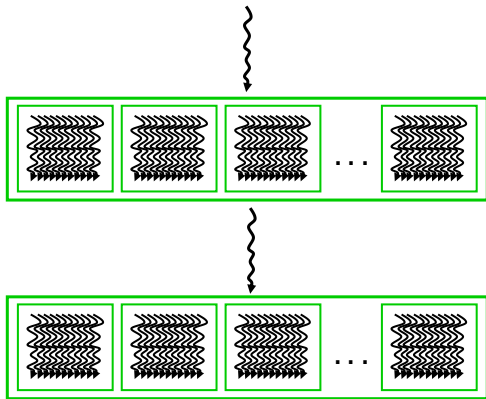
Parallel Kernel (device)

```
KernelA<<<nBlk, nThr>>>(args);
```

Serial Code (host)

Parallel Kernel (device)

```
KernelB<<<nBlk, nThr>>>(args);
```



# Sample GPU SIMT Code (Simplified)

---

CPU code

```
for (ii = 0; ii < 100000; ++ii) {  
    C[ii] = A[ii] + B[ii];  
}
```



CUDA code

```
// there are 100000 threads  
__global__ void KernelFunction(...) {  
    int tid = blockDim.x * blockIdx.x + threadIdx.x;  
    int varA = aa[tid];  
    int varB = bb[tid];  
    C[tid] = varA + varB;  
}
```

# Sample GPU Program (Less Simplified)

## CPU Program

```
void add_matrix  
( float *a, float* b, float *c, int N) {  
    int index;  
    for (int i = 0; i < N; ++i)  
        for (int j = 0; j < N; ++j) {  
            index = i + j*N;  
            c[index] = a[index] + b[index];  
        }  
}  
  
int main () {  
  
    add_matrix (a, b, c, N);  
}
```

## GPU Program

```
__global__ add_matrix  
( float *a, float *b, float *c, int N) {  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
    int j = blockIdx.y * blockDim.y + threadIdx.y;  
    int index = i + j*N;  
    if (i < N && j < N)  
        c[index] = a[index]+b[index];  
}  
  
int main() {  
    dim3 dimBlock( blocksize, blocksize );  
    dim3 dimGrid (N/dimBlock.x, N/dimBlock.y);  
    add_matrix<<<dimGrid, dimBlock>>>( a, b, c, N);  
}
```

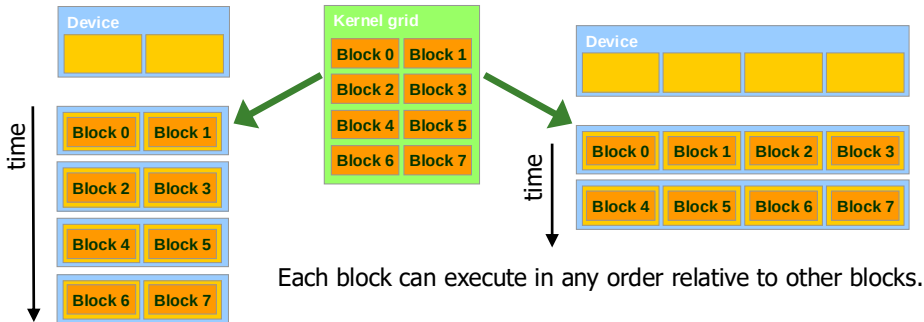
# CUDA/OpenCL Programming Model

---

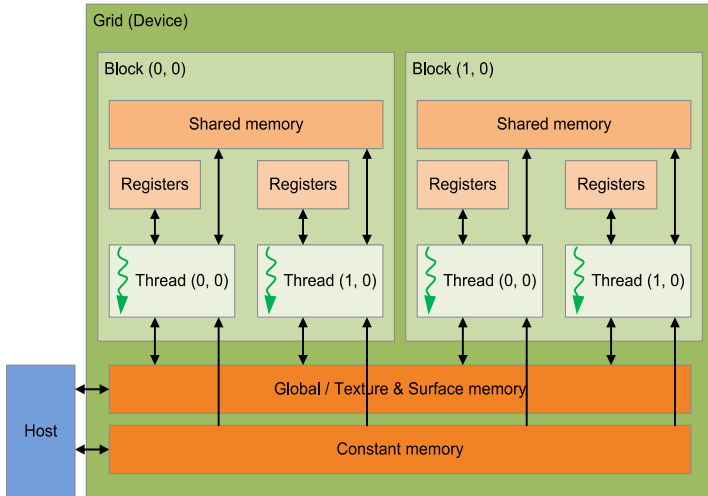
- SIMT or SPMD
- Bulk synchronous programming
  - Global (coarse-grain) synchronization between kernels
- The host (typically CPU) allocates memory, copies data, and launches kernels
- The device (typically GPU) executes kernels
  - Grid (NDRange)
  - Block (work-group)
    - Within a block, shared memory, and synchronization
  - Thread (work-item)

# Transparent Scalability

- Hardware is **free to schedule** thread blocks



# Memory Hierarchy



# Traditional Program Structure in CUDA

## ■ Function prototypes

```
float serialFunction(...);  
__global__ void kernel(...);
```

## ■ main()

- ❑ 1) **Allocate memory** space on the device – `cudaMalloc(&d_in, bytes);`
- ❑ 2) Transfer data from **host to device** – `cudaMemcpy(d_in, h_in, ...);`
- ❑ 3) Execution configuration setup: #blocks and #threads
- ❑ 4) **Kernel call** – `kernel<<<execution configuration>>>(args...);`
- ❑ 5) Transfer results from **device to host** – `cudaMemcpy(h_out, d_out, ...);`

repeat  
as needed



## ■ Kernel – `__global__ void kernel(type args,...)`

- ❑ Automatic variables transparently assigned to **registers**
- ❑ **Shared memory**: `__shared__`
- ❑ Intra-block **synchronization**: `__syncthreads();`

# CUDA Programming Language

---

- **Memory allocation**

```
cudaMalloc((void**)&d_in, #bytes);
```

- **Memory copy**

```
cudaMemcpy(d_in, h_in, #bytes, cudaMemcpyHostToDevice);
```

- **Kernel launch**

```
kernel<<< #blocks, #threads >>>(args);
```

- **Memory deallocation**

```
cudaFree(d_in);
```

- **Explicit synchronization**

```
cudaDeviceSynchronize();
```

# Indexing and Memory Access: 1D Grid

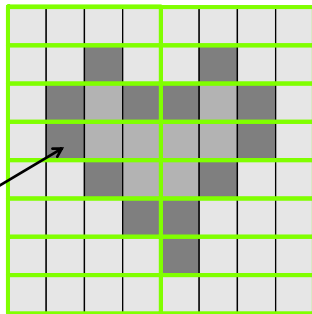
- One GPU thread per pixel
- Grid of Blocks of Threads

- `gridDim.x`, `blockDim.x`
- `blockIdx.x`, `threadIdx.x`

`blockIdx.x`

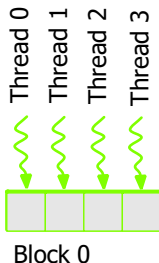
`threadIdx.x`

Block 0



$$6 * 4 + 1 = 25$$

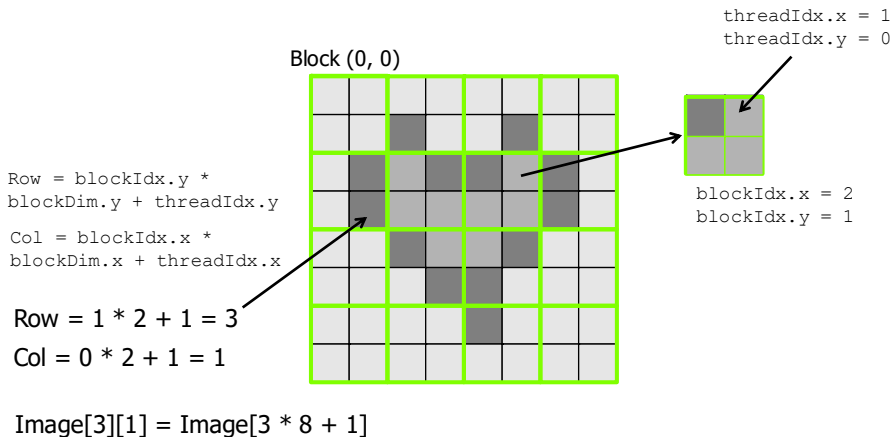
`blockIdx.x * blockDim.x + threadIdx.x`



# Indexing and Memory Access: 2D Grid

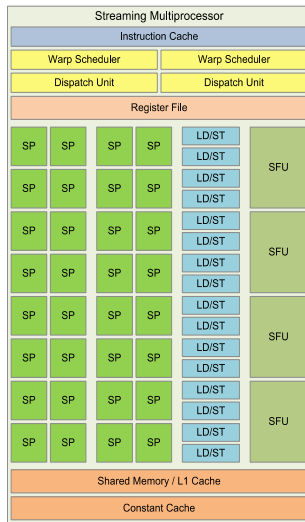
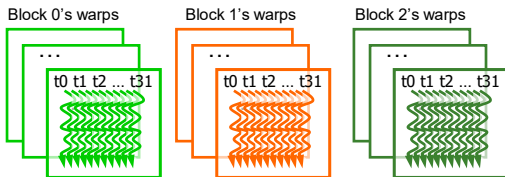
## ■ 2D blocks

□ `gridDim.x`, `gridDim.y`

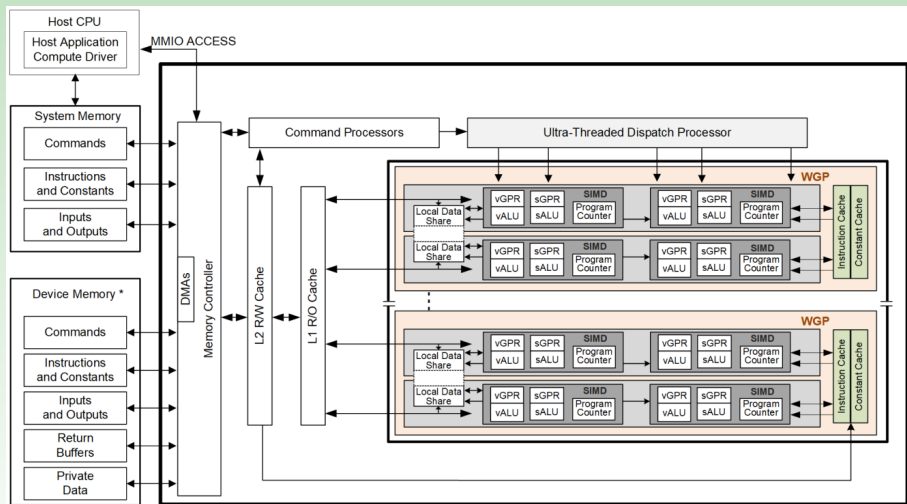


# From Blocks to Warps

- GPU cores: SIMD pipelines
  - ❑ Streaming Multiprocessors (SM)
  - ❑ Streaming Processors (SP)
- Blocks are divided into warps
  - ❑ SIMD unit (32 threads)



NVIDIA Fermi architecture



Źródło: "RDNA4" Instruction Set Architecture Reference Guide

# Warp-based SIMD vs. Traditional SIMD

---

- Traditional SIMD contains a single thread
  - Sequential instruction execution; lock-step operations in a SIMD instruction
  - Programming model is SIMD (no extra threads) → SW needs to know vector length
  - ISA contains vector/SIMD instructions
- Warp-based SIMD consists of multiple scalar threads executing in a SIMD manner (i.e., same instruction executed by all threads)
  - Does not have to be lock step
  - Each thread can be treated individually (i.e., placed in a different warp)  
→ programming model not SIMD
    - SW does not need to know vector length
    - Enables multithreading and flexible dynamic grouping of threads
  - ISA is scalar → SIMD operations can be formed dynamically
  - Essentially, it is SPMD programming model implemented on SIMD hardware

# SPMD

---

- Single procedure/program, multiple data
  - This is a programming model rather than computer organization
- Each processing element executes the same procedure, except on different data elements
  - Procedures can synchronize at certain points in program, e.g. barriers
- Essentially, multiple instruction streams execute the same program
  - Each program/procedure 1) works on different data, 2) can execute a different control-flow path, at run-time
  - Many scientific applications are programmed this way and run on MIMD hardware (multiprocessors)
  - Modern GPUs programmed in a similar way on a SIMD hardware

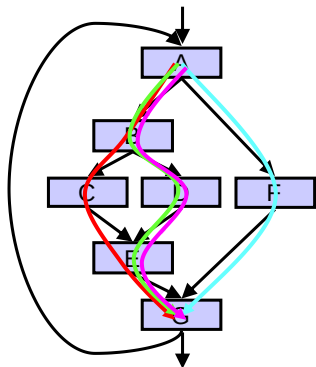
# SIMD vs. SIMT Execution Model

---

- SIMD: A single **sequential instruction stream** of SIMD **instructions** → each instruction specifies multiple data inputs
  - [VLD, VLD, VADD, VST], VLEN
- SIMT: **Multiple instruction streams** of **scalar instructions** → threads grouped dynamically into warps
  - [LD, LD, ADD, ST], NumThreads
- Two Major SIMT Advantages:
  - **Can treat each thread separately** → i.e., can execute each thread independently on any type of scalar pipeline → MIMD processing
  - **Can group threads into warps flexibly** → i.e., can group threads that are supposed to *truly* execute the same instruction → dynamically obtain and maximize benefits of SIMD processing

# Threads Can Take Different Paths in Warp-based SIMD

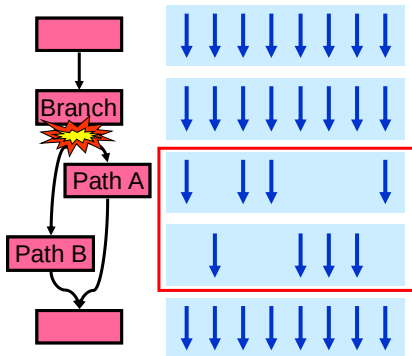
- Each thread can have **conditional control flow instructions**
- Threads can execute different control flow paths



| Thread Warp |          |          |          | Common PC |
|-------------|----------|----------|----------|-----------|
| Thread 1    | Thread 2 | Thread 3 | Thread 4 |           |

# Control Flow Problem in GPUs/SIMT

- A GPU uses a SIMD pipeline to save area on control logic
  - Groups scalar threads into warps
- **Branch divergence** occurs when threads inside warps branch to different execution paths



**This is the same as conditional/predicated/masked execution.  
Recall the Vector Mask and Masked Vector Operations?**

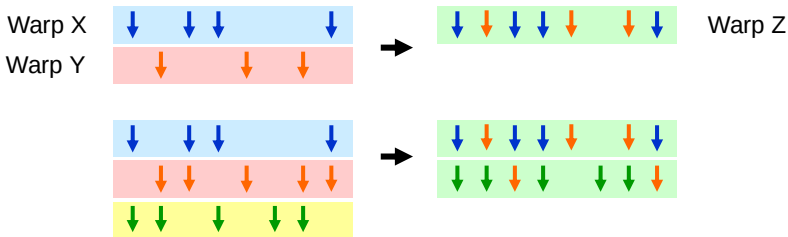
# Remember: Each Thread Is Independent

---

- Two Major SIMT Advantages:
  - Can treat each thread separately → i.e., can execute each thread independently on any type of scalar pipeline → MIMD processing
  - Can group threads into warps flexibly → i.e., can group threads that are supposed to *truly* execute the same instruction → dynamically obtain and maximize benefits of SIMD processing
  
- If we have many threads
- We can find individual threads that are at the same PC
- And, group them together into a single warp dynamically
- This reduces “divergence” → improves SIMD utilization
  - SIMD utilization: fraction of SIMD lanes executing a useful operation (i.e., executing an active thread)

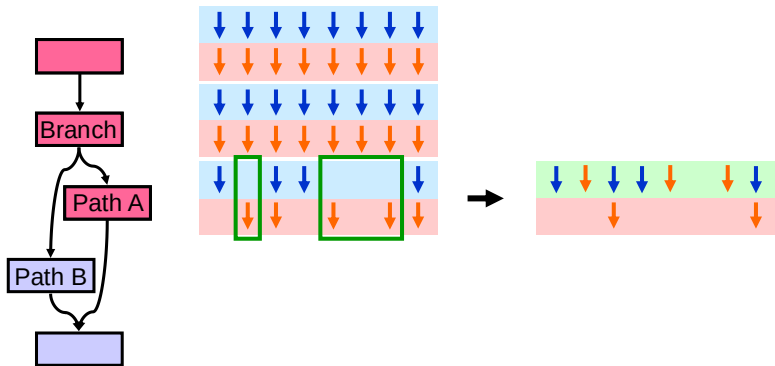
# Dynamic Warp Formation/Merging

- Idea: Dynamically merge threads executing the same instruction (after branch divergence)
- Form new warps from warps that are waiting
  - Enough threads branching to each path enables the creation of full new warps



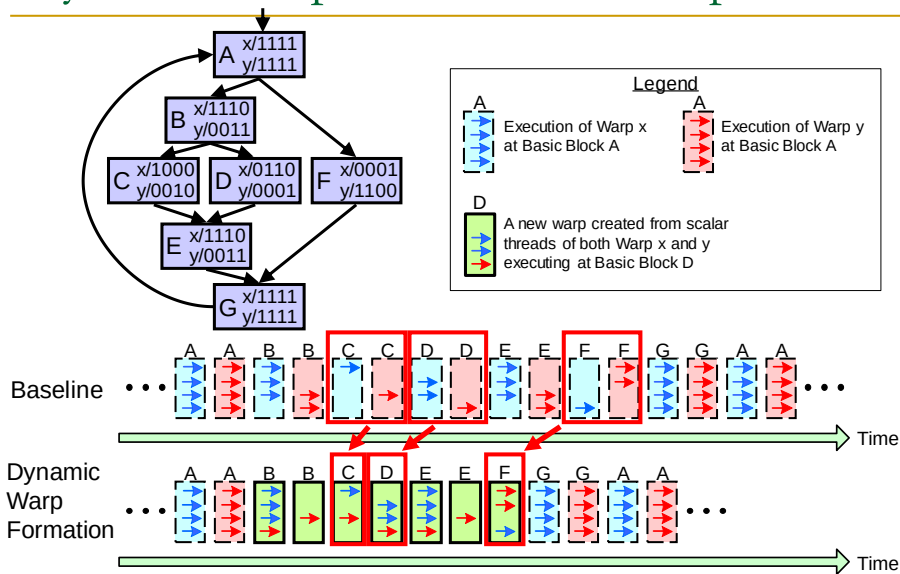
# Dynamic Warp Formation/Merging

- Idea: Dynamically merge threads executing the same instruction (after branch divergence)

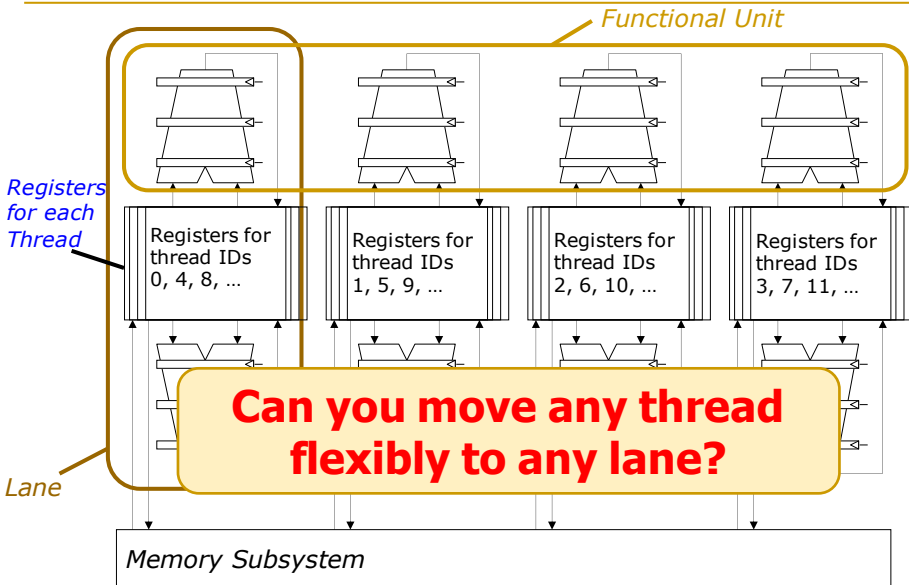


- Fung et al., “Dynamic Warp Formation and Scheduling for Efficient GPU Control Flow,” MICRO 2007.

# Dynamic Warp Formation Example



# Hardware Constraints Limit Flexibility of Warp Grouping



## Example CPU ISA Registers

Scalar Registers (64-bit)

|          |
|----------|
| R0 / RAX |
| R1 / RCX |
| R2 / RDX |
| R3 / RBX |
| R4 / RSP |
| R5 / RBP |
| R6 / RSI |
| R7 / RDI |
| R8       |
| ...      |
| R15      |

Instruction Pointer

|     |
|-----|
| RIP |
|-----|

Flags

|   |   |   |   |   |   |   |   |   |   |                  |
|---|---|---|---|---|---|---|---|---|---|------------------|
| C | I | F | A | Z | S | T | O | D | O | etc, other stuff |
|---|---|---|---|---|---|---|---|---|---|------------------|

Model Specific Registers



Vector Registers (512-bit)

|      |
|------|
| ZMM0 |
| ZMM1 |
| ZMM2 |

...

|       |
|-------|
| ZMM31 |
|-------|

Mask Registers (64-bit)

|    |
|----|
| K0 |
| K1 |

...

|    |
|----|
| K7 |
|----|

## RDNA 4 Registers

Scalar Registers (32-bit)

|    |
|----|
| S0 |
| S1 |
| S2 |

...

|               |
|---------------|
| S105          |
| S106 / VCC_HI |
| S107 / VCC_LO |
| S108 / TTMP0  |

...

|                |
|----------------|
| S123 / TTMP15  |
| S124 / NULL    |
| S125 / M0      |
| S126 / EXEC_LO |
| S127 / EXEC_HI |

Vector Registers (1024-bit)

|    |
|----|
| v0 |
| v1 |
| v2 |

...

|          |
|----------|
| v95      |
| v96 = v0 |

...

|           |
|-----------|
| v255 = v0 |
|-----------|

VGPRs beyond the allocated range have varying useless behavior, like aliasing to v0 in most read cases, turning writes into NOPs, or reading from  $v(n \% 4)$  for VOPD instructions

For example, if the compiler/driver allocated 96 VGPRs

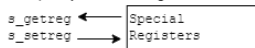
Instruction Pointer (Program Counter)

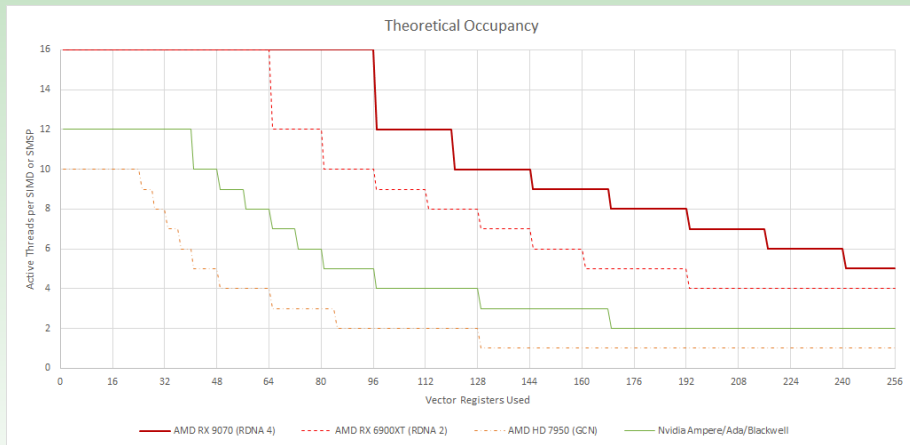
|    |
|----|
| PC |
|----|

Scalar Condition Code

|     |
|-----|
| SCC |
|-----|

Infrequently Accessed Registers





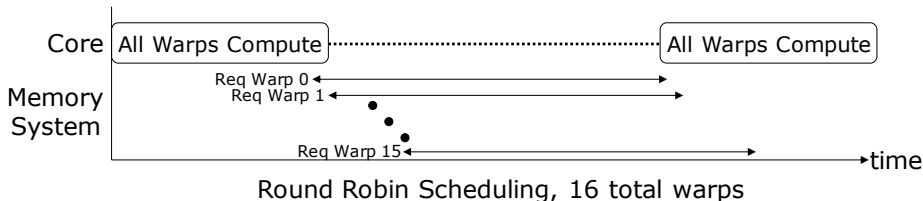
Źródło: <https://chipsandcheese.com/p/dynamic-register-allocation-on-amds>

|      | Opcode              | Operands                                                                                   | Hit count | Instruction cost (%) | Latency |
|------|---------------------|--------------------------------------------------------------------------------------------|-----------|----------------------|---------|
|      | BBF0_117            |                                                                                            |           |                      |         |
| 5572 | s_delay_alu         | instid0(VALU_DEP_1)   instskip(NEXT)   instid1(VALU_DEP_1)                                 | 1040      | 0.00                 |         |
| 5573 | v_max_u32_dpp       | v5, v5, v5 row_shr:1 row_mask:0xf bank_mask:0xf                                            | 1040      | 0.03                 | 12 clk  |
| 5574 | v_max_u32_dpp       | v5, v5, v5 row_shr:2 row_mask:0xf bank_mask:0xf                                            | 1040      | 0.02                 | 11 clk  |
| 5575 | s_delay_alu         | instid0(VALU_DEP_1)   instskip(NEXT)   instid1(VALU_DEP_1)                                 | 1040      | 0.00                 |         |
| 5576 | v_max_u32_dpp       | v5, v5, v5 row_shr:4 row_mask:0xf bank_mask:0xf                                            | 1040      | 0.03                 | 11 clk  |
| 5577 | v_max_u32_dpp       | v5, v5, v5 row_shr:8 row_mask:0xf bank_mask:0xf                                            | 1040      | 0.03                 | 12 clk  |
| 5578 | s_delay_alu         | instid0(VALU_DEP_1)   instskip(NEXT)   instid1(VALU_DEP_1)                                 | 1040      | 0.00                 |         |
| 5579 | v_permmix6_b32      | v6, v5, -1, -1 op_sel:[1, 0]                                                               | 1040      | 0.03                 | 12 clk  |
| 5580 | v_max_u32_e32       | v6, v6, v5                                                                                 | 1040      | 0.01                 | 6 clk   |
| 5581 | s_delay_alu         | instid0(VALU_DEP_1)   instskip(SKIP_1)   instid1(VALU_DEP_1)                               | 1040      | 0.00                 |         |
| 5582 | v_readlane_b32      | s3, v6, 31                                                                                 | 1040      | 0.02                 | 10 clk  |
| 5583 | v_cndmask_b32_e32   | v5, -1, v3, vcc_lo                                                                         | 1040      | 0.03                 | 11 clk  |
| 5584 | v_cmpx_eq_u32_e64   | s3, v5                                                                                     | 1040      | 0.06                 | 27 clk  |
| 5585 | v_readfirstlane_b32 | s4, v2                                                                                     | 1040      | 0.07                 | 30 clk  |
| 5586 | s_wait_alu          | depctr_va_sdst(0)                                                                          | 1040      | 0.00                 |         |
| 5587 | s_delay_alu         | instid0(VALU_DEP_1)                                                                        | 1040      | 0.00                 |         |
| 5588 | v_cmpx_eq_u32_e64   | s4, v2                                                                                     | 1040      | 0.00                 | 1 clk   |
| 5589 | s_wait_kmcnt        | 0x0                                                                                        | 1040      | 0.01                 | 6 clk   |
| 5590 | s_and_b32           | s6, s3, 0xff mask with 0xFF, truncating value to 255 (max VGPR count)                      | 1040      | 0.02                 | 11 clk  |
| 5591 | s_wait_alu          | depctr_sa_sdst(0)                                                                          | 1040      | 0.00                 |         |
| 5592 | s_max_i32           | s6, s6, s101 not sure where s101 comes from, but it could be a minimum register allocation | 1040      | 0.09                 | 40 clk  |
| 5593 | s_wait_alu          | depctr_sa_sdst(0)                                                                          | 1040      | 0.00                 |         |
| 5594 | s_allloc_vgpr       | s6 ask for register count specified in scalar GPR s6                                       | 1040      | 0.04                 | 18 clk  |
| 5595 | s_cbranch_scc0      | _L101 if SCC=0, allocation failed                                                          | 1040      | 0.01                 | 5 clk   |
| 5596 | _L103               |                                                                                            |           |                      |         |
| 5597 | ds_nop              | SCC=1 (allocation success) case                                                            | 1040      | 0.00                 | 1 clk   |
| 5598 | ds_nop              |                                                                                            | 1040      | 0.00                 | 1 clk   |
| 5599 | ds_nop              |                                                                                            | 1040      | 0.00                 | 1 clk   |
| 5600 | ds_nop              |                                                                                            | 1040      | 0.00                 | 1 clk   |
| 5601 | ds_nop              |                                                                                            | 1040      | 0.00                 | 1 clk   |
| 5602 | ds_nop              |                                                                                            | 1040      | 0.00                 | 2 clk   |
| 5603 | ds_nop              |                                                                                            | 1040      | 0.00                 | 1 clk   |
| 5604 | ds_nop              |                                                                                            | 1040      | 0.00                 | 2 clk   |
| 5605 | s_wait_alu          | depctr_sa_sdst(0) depctr_va_vcc(0)                                                         | 0         | 0.00                 |         |
| 5606 | s_setpc_b64         | s[4:5] Function call to intersection/hit/miss shader                                       | 1040      | 0.33                 | 152 clk |
| 5607 | _L101               |                                                                                            |           |                      |         |
| 5608 | s_allloc_vgpr       | s101 Set allocation back to a potentially lower value? Not sure why it's necessary         | 194       | 0.05                 | 120 clk |
| 5609 | _L102               |                                                                                            |           |                      |         |
| 5610 | s_sleep             | 2                                                                                          | 23367     | 0.11                 | 2 clk   |
| 5611 | s_allloc_vgpr       | s6 Keep trying to get the requested VGPR allocation in a busy wait loop                    | 23367     | 0.68                 | 13 clk  |
| 5612 | s_cbranch_scc0      | _L102                                                                                      | 23367     | 5.00                 | 118 clk |
| 5613 | BBF0_118            |                                                                                            |           |                      |         |
| 5614 | s_branch            | _L103 Until it succeeds, then go to the function call                                      | 194       | 0.01                 | 33 clk  |

Źródło: <https://chipsandcheese.com/p/dynamic-register-allocation-on-amds>

# Large Warps and Two-Level Warp Scheduling

- Two main reasons for GPU resources be underutilized
  - ❑ Branch divergence
  - ❑ Long latency operations



# Large Warp Microarchitecture Example

- Reduce **branch divergence** by having large warps
- Dynamically break down a large warp into sub-warps

Decode Stage

|   |   |   |   |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |

Sub-warp 0 mask

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
|---|---|---|---|

Sub-warp 0 mask

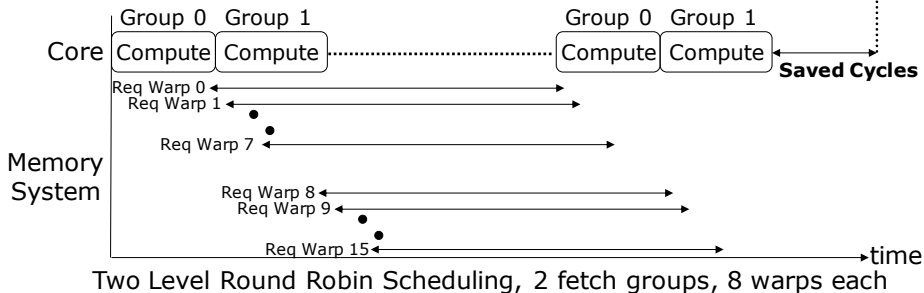
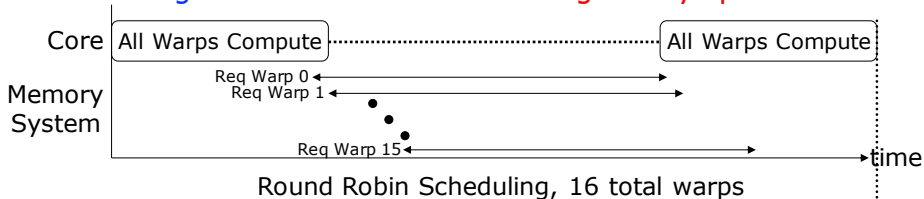
|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
|---|---|---|---|

Sub-warp 0 mask

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
|---|---|---|---|

# Two-Level Round Robin

- Scheduling in two levels to deal with long latency operations



# An Example GPU

# NVIDIA GeForce GTX 285

---

- NVIDIA-speak:

- 240 stream processors
- “SIMT execution”

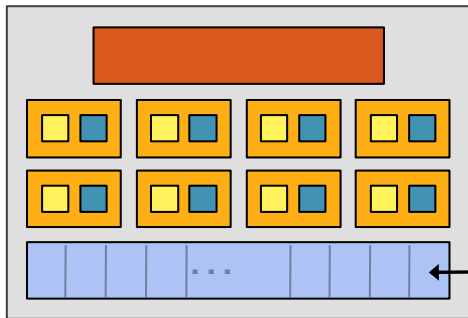
- Generic speak:

- 30 cores
- 8 SIMD functional units per core



- NVIDIA, “[NVIDIA GeForce GTX 200 GPU. Architectural Overview. White Paper,](#)” 2008.

# NVIDIA GeForce GTX 285 “core”



64 KB of storage  
for thread contexts  
(registers)



= SIMD functional unit, control  
shared across 8 units



= instruction stream decode

■ = multiply-add

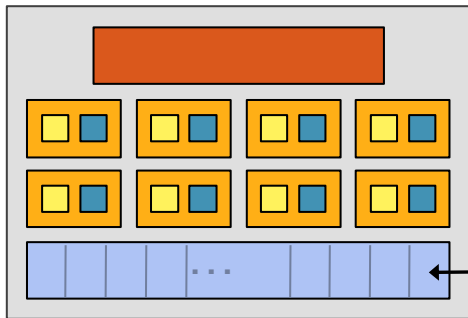
■ = multiply



= execution context storage

# NVIDIA GeForce GTX 285 “core”

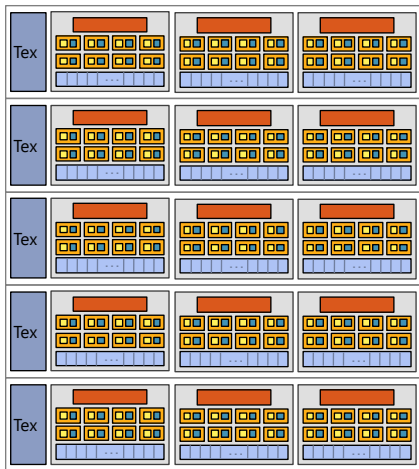
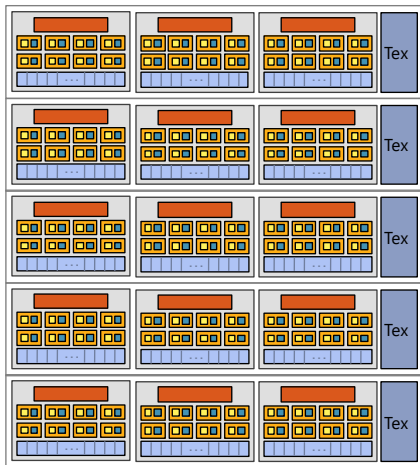
---



64 KB of storage  
for thread contexts  
(registers)

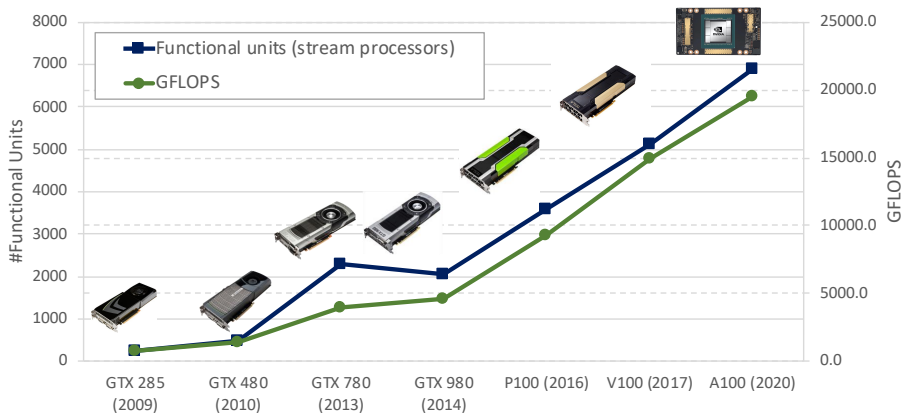
- Groups of 32 **threads** share instruction stream (each group is a Warp)
- Up to 32 warps are simultaneously interleaved
- Up to 1024 thread contexts can be stored

# NVIDIA GeForce GTX 285

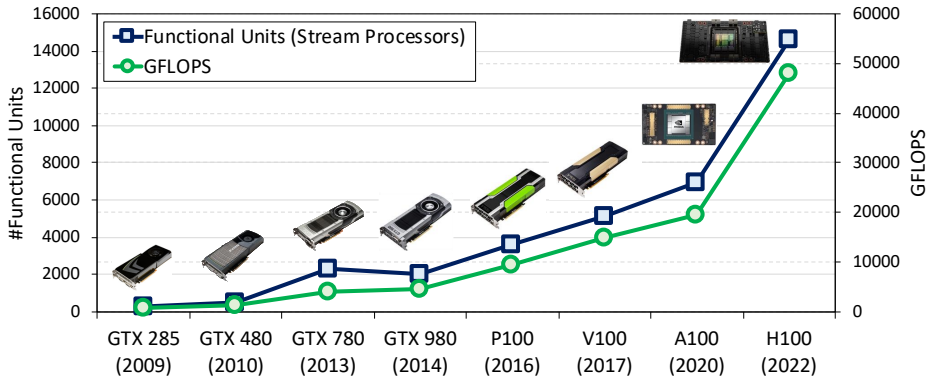


30 cores on the GTX 285: 30,720 threads

# Evolution of NVIDIA GPUs



# Evolution of NVIDIA GPUs (Updated)



# NVIDIA V100

---

- NVIDIA-speak:
  - ❑ 5120 stream processors
  - ❑ “SIMT execution”
- Generic speak:
  - ❑ 80 cores
  - ❑ 64 SIMD functional units per core
  - ❑ Tensor cores for Machine Learning
- NVIDIA, “[NVIDIA Tesla V100 GPU Architecture. White Paper](#),” 2017.



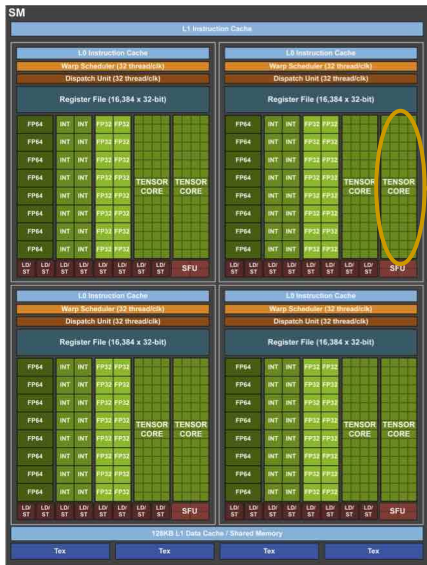
# NVIDIA V100 Block Diagram



<https://devblogs.nvidia.com/inside-volta/>

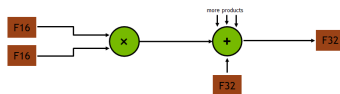
80 cores on the V100

# NVIDIA V100 Core



15.7 TFLOPS Single Precision  
7.8 TFLOPS Double Precision  
125 TFLOPS for Deep Learning (Tensor cores)

FP16 storage/input      Full precision product      Sum with FP32 accumulator      Convert to FP32 result

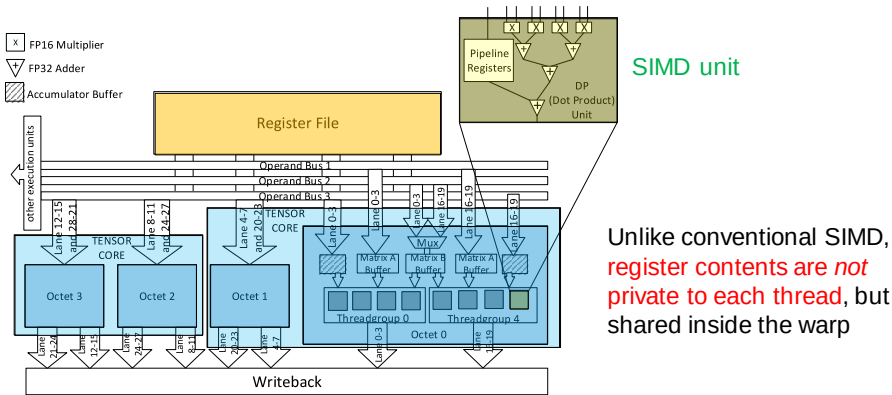


$$D = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{pmatrix} \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{pmatrix} + \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{pmatrix}$$

FP16 or FP32      FP16      FP16      FP16 or FP32

# Tensor Core Microarchitecture (Volta)

- Each warp utilizes **two tensor cores**
- Each tensor core contains **two “octets”**
  - **16 SIMD units per tensor core** (8 per octet)
  - **4x4 matrix-multiply and accumulate each cycle per tensor core**

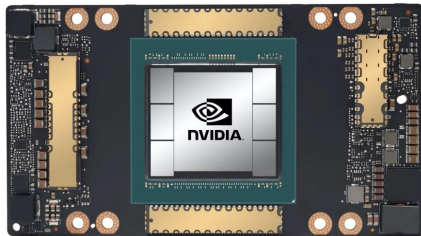


Proposed\* tensor core microarchitecture

# NVIDIA A100

---

- NVIDIA-speak:
  - ❑ 6912 stream processors
  - ❑ “SIMT execution”
- Generic speak:
  - ❑ 108 cores
  - ❑ 64 SIMD functional units per core
  - ❑ Tensor cores for Machine Learning
    - Support for sparsity
    - New floating point data type (TF32)





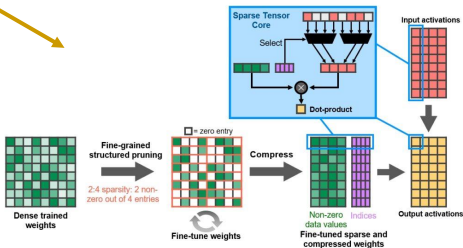
# NVIDIA A100 Core



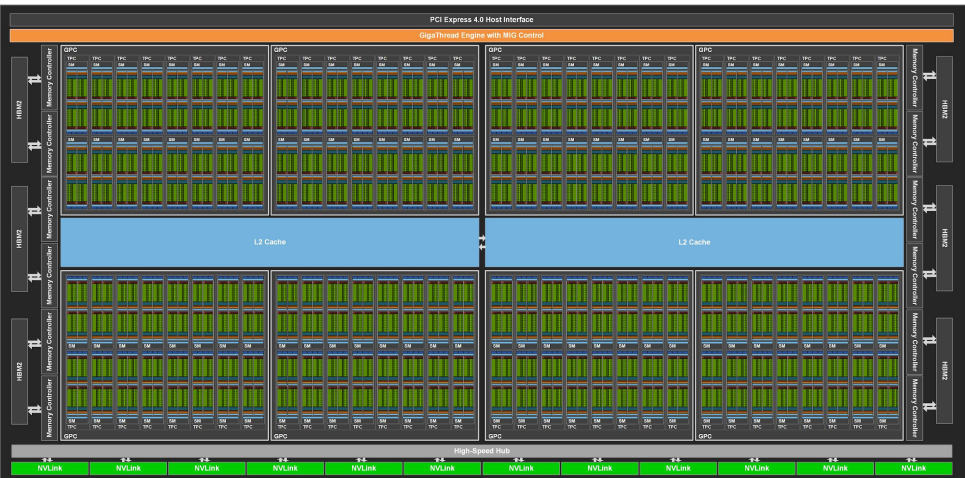
19.5 TFLOPS Single Precision

9.7 TFLOPS Double Precision

312 TFLOPS for Deep Learning (Tensor cores)



# NVIDIA A100 Block Diagram



<https://developer.nvidia.com/blog/nvidia-ampere-architecture-in-depth/>

108 cores on the A100  
(Up to 128 cores in the full-blown chip)

40MB L2 cache

# NVIDIA H100 Block Diagram



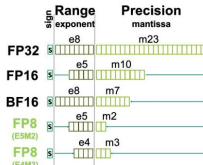
<https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>

144 cores on the full GH100  
60MB L2 cache

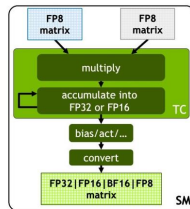
# NVIDIA H100 Core



48 TFLOPS Single Precision\*  
 24 TFLOPS Double Precision\*  
 800 TFLOPS (FP16, Tensor Cores)\*



Allocate 1 bit to either  
range or precision



Support for multiple accumulator  
and output types

# Clarification of Some GPU Terms

---

| Generic Term                                | NVIDIA Term                                                           | AMD Term       | Comments                                                                     |
|---------------------------------------------|-----------------------------------------------------------------------|----------------|------------------------------------------------------------------------------|
| Vector length                               | Warp size                                                             | Wavefront size | Number of threads that run in parallel (lock-step) on a SIMD functional unit |
| Pipelined functional unit / Scalar pipeline | Streaming processor / CUDA core                                       | -              | Functional unit that executes instructions for one GPU thread                |
| SIMD functional unit / SIMD pipeline        | Group of N streaming processors (e.g., N=8 in GTX 285, N=16 in Fermi) | Vector ALU     | SIMD functional unit that executes instructions for an entire warp           |
| GPU core                                    | Streaming multiprocessor                                              | Compute unit   | It contains one or more warp schedulers and one or several SIMD pipelines    |

# Czasami rzeczy są implementowane zaskakująco późno

## DEEPER DIVE

### OUT OF ORDER MEMORY

#### Latency of memory requests can be critical for performance

- We see raytracing workloads as highly sensitive to this factor
- Traversal through the BVH structure, Texture and buffer requests for shading etc.

#### RDNA 4 introduces additional out-of-order queues for memory requests

- RDNA 3 data returns were ordered with respect to when the requests were made
- Data items returning quickly could be delayed behind long-latency requests

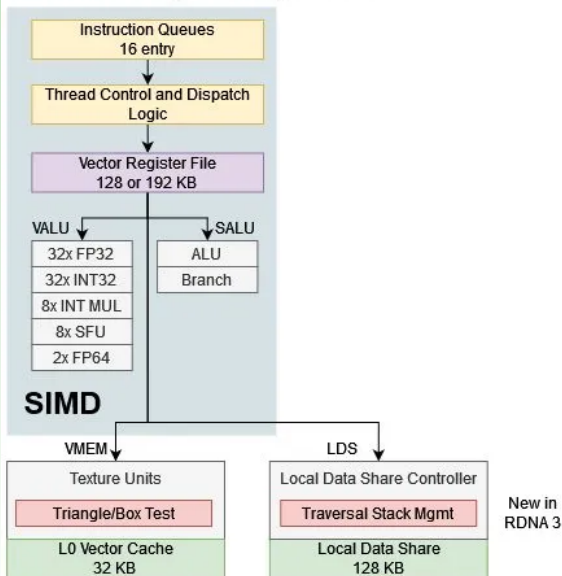
#### RDNA 4 allows requests from different shaders to be satisfied out-of-order

- This allows shaders to execute efficiently regardless of some long latency requests  
E.g. an access to an un-cached leaf node may no longer hold up surface shading

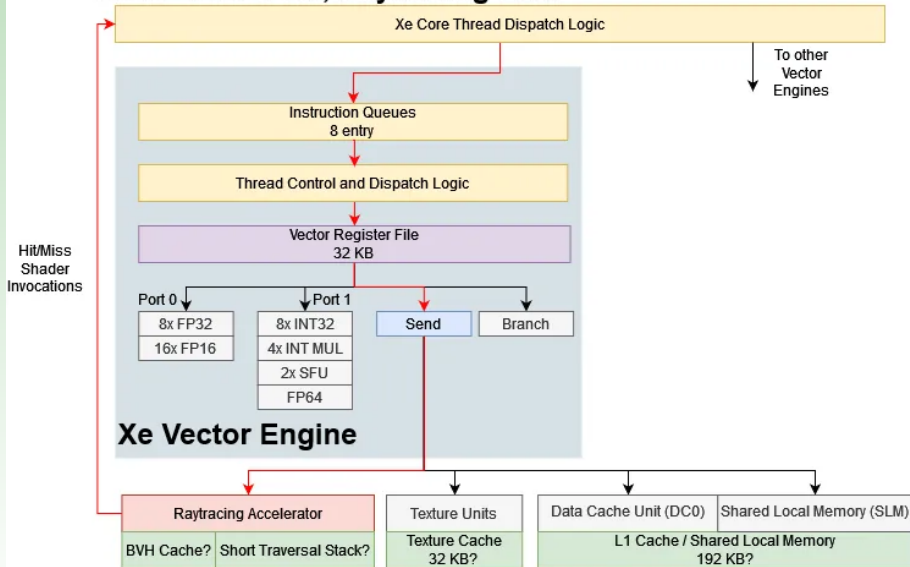
#### This change increases RDNA 4 performance for many workloads



# RDNA 3 Raytracing Flow



# Xe-LPG Xe Core, Raytracing Flow



Źródło: <https://chipsandcheese.com/p/raytracing-on-meteor-lakes-igpu>



AMD (kw. 2025). *"RDNA4" Instruction Set Architecture Reference Guide*. URL:

<https://www.amd.com/content/dam/amd/en/documents/radeon-tech-docs/instruction-set-architectures/rdna4-instruction-set-architecture.pdf>.